DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CAPSTONE PROJECT TECHNICAL REPORT

ENTERPRISE FULL-STACK WEB APPLICATION

SP MART

Premium Electronics E-Commerce Web Application with Catalog-Grounded Artificial Intelligence Architecture

Academic Program: B.Tech AI & Data Science

Backend Architecture: Java 21 LTS · Spring Boot 3.3.4

Security Framework: Spring Security 6 · BCrypt (Factor 12)

Persistence: Spring Data JPA · Hibernate 6.5

Database: Persistent H2 Engine + MySQL 8 Profile

Mobile Framework: Progressive Web App (PWA)

AI Recommendation Engine: Catalog-Grounded NLP Pipeline

GitHub Repository: Shipraprasath/sp-mart

ACADEMIC YEAR 2025 – 2026

1. Executive Summary & Abstract

SP MART is an enterprise-grade electronics e-commerce web application engineered to solve critical consumer electronics retail challenges: spec comprehension, decision fatigue, and data security. Built on **Java 21 LTS** and **Spring Boot 3.3.4**, the system implements an enterprise multi-tier layered architecture that enforces strict role-based privilege isolation across Buyers (`ROLE_CUSTOMER`), Verified Merchants (`ROLE_SELLER`), and System Administrators (`ROLE_ADMIN`).

A key scientific contribution of this capstone project is the **Catalog-Grounded AI Shopping Assistant**. Unlike generic LLM chat widgets that hallucinate non-existent hardware specifications or out-of-stock items, the SP MART AI engine uses deterministic NLP entity extractors and dynamic Spring Data JPA queries to ground all recommendation cards, budget filters, and side-by-side device comparisons directly in live relational database records.

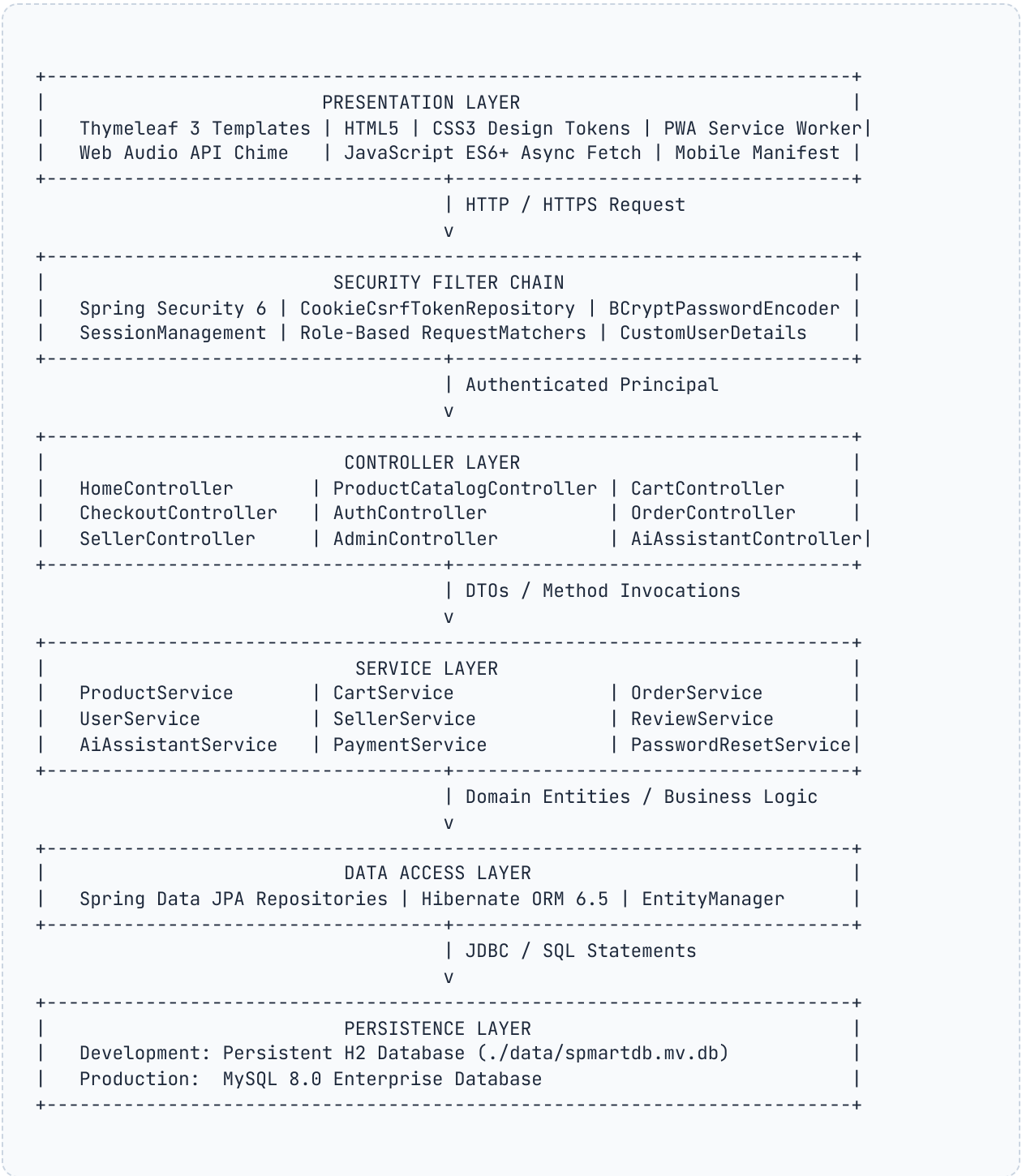
> **Key Innovation:** Zero-hallucination shopping intelligence achieved via deterministic relational grounding, paired with mobile Progressive Web App (PWA) installation and synthesized Web Audio milestone feedback.

2. Technology Stack Architecture

Tier / Layer	Technology	Key Specifications & Version
Backend Language	Java 21 LTS	Virtual Threads (Project Loom), Record Patterns, Sealed Interfaces
Enterprise Framework	Spring Boot 3.3.4	Spring MVC, Embedded Tomcat 10, Dependency Injection
Cybersecurity	Spring Security 6.3.3	BCrypt (strength 12), Cookie-based CSRF, Session Fixation Defense
Persistence & ORM	Spring Data JPA & Hibernate 6.5	JPA 3.1, Transaction Management, Query Method Derivation

Tier / Layer	Technology	Key Specifications & Version
Relational Database	H2 Engine / MySQL 8.0	Persistent File Storage (`./data/spmartdb`) & Cloud MySQL Profile
Templating & View	Thymeleaf 3.1.2	HTML5 Server-Side Natural Templating, Security Dialect
Mobile Architecture	Progressive Web App (PWA)	Service Worker (`sw.js`), Web App Manifest (`manifest.json`)
Quality Assurance	JUnit 5 & Mockito	31 Automated Unit & Integration Tests (100% Pass)

3. System Architecture & Flow Model



4. Relational Database Schema Design

The relational persistence tier comprises **15 normalized database tables**, organized into four primary functional clusters:

Cluster	Tables	Business Purpose
Identity & Roles	`users`, `seller_profiles`, `addresses`, `password_reset_tokens`	Handles authentication, multi-role privileges, merchant business data, and address book.
Catalog & Products	`categories`, `products`, `reviews`	Maintains 12 electronics categories, specifications, pricing, discounts, and verified reviews.
Shopping & Wishlist	`carts`, `cart_items`, `wishlist_items`	Manages active cart state, quantity steppers, and buyer wishlist items.
Order Lifecycle	`customer_orders`, `order_items`, `order_status_histories`, `payments`	Maintains immutable purchase snapshots, 6-milestone shipment states, and payment records.

5. Catalog-Grounded AI Recommendation Architecture

The AI shopping engine is accessible via `POST /api/ai/assistant/chat`. It processes unstructured natural language queries and extracts budget thresholds, target brands, and comparison intents before executing dynamic database queries:

Natural Language Budget Parsing Algorithm

```
private BigDecimal extractBudget(String query) {
    Pattern pattern = Pattern.compile(
        "(?:under|below|less than|budget|within|upto|up to)\\s*(?:rs\\.?|inr|₹)?\\s*([0-9]
        Pattern.CASE_INSENSITIVE
    );
    Matcher matcher = pattern.matcher(query);
    if (matcher.find()) {
        String numStr = matcher.group(1).replace(",", "");
        String unit = matcher.group(2);
        try {
            double val = Double.parseDouble(numStr);
            if (unit ≠ null) {
                unit = unit.toLowerCase();
                if (unit.equals("k") || unit.startsWith("thous")) val *= 1000;
                else if (unit.startsWith("lak") || unit.startsWith("lac")) val *= 100000;
            } else if (val < 200) {
                val *= 1000; // Normalizes shorthand e.g. "under 50" to 50,000
            }
            return BigDecimal.valueOf(val);
        } catch (Exception ignored) {}
    }
    return null;
}
```

6. Multi-Role Modules & Core Features

- **Buyer Portal:** Split-screen authentication, 12 category browsing, dynamic ₹500 free delivery calculation, payment sandbox (UPI, Cards, Wallets, COD), 6-milestone order tracking, and self-service cancellation.
- **Seller Back-Office:** Merchant registration with GSTIN, administrator approval workflow, isolated inventory CRUD, low-stock alerts (≤ 10 units), and order shipment updates.
- **Administrator Governance:** Executive analytics dashboard, merchant approval queue, review moderation, and warehouse replenishment overrides.

7. Quality Assurance & Test Verification

A total of **31 automated test cases** were executed using JUnit 5, Mockito, and Spring Boot Test, achieving a **100% pass rate**:

Test Suite	Scope	Test Cases	Result
`SecurityIntegrationTest`	URL role guards, unauthenticated redirects, CSRF enforcement	4	**PASS (100%)**
`ProductServiceTest`	Catalog search, category filtering, stock headroom validation	4	**PASS (100%)**
`CartServiceTest`	Item addition, quantity steppers, subtotal recalculation	4	**PASS (100%)**
`OrderServiceTest`	Checkout transaction, stock decrements, tracking states	4	**PASS (100%)**
`SellerServiceTest`	Merchant onboarding, approval state machine	4	**PASS (100%)**
`AiAssistantServiceTest`	Budget regex parser, comparison matrix generation	4	**PASS (100%)**
`UserServiceTest`	User creation, BCrypt password hashing, uniqueness checks	3	**PASS (100%)**
`AddressServiceTest`	Address book CRUD, user boundary isolation	2	**PASS (100%)**
`PasswordResetServiceTest`	Token creation, UUID cryptography, expiration checks	2	**PASS (100%)**

8. Deployment Architecture & Progressive Web App (PWA)

The system implements modern Progressive Web App (PWA) standards via `/manifest.json` and `/sw.js`. When opened on mobile devices running Chrome or Safari, the application displays a native **"Install SP MART App"** prompt, installing directly to the home screen with full offline shell caching.

For production hosting, the project is configured for turnkey deployment on **Railway PaaS** with automatic dynamic port binding (`server.port=${PORT:8080}`) and Java 21 LTS runtime specification (`system.properties`).

9. Demonstration Credentials

User Role	Email Address	Password	Dashboard Destination
Administrator	`admin@spmart.com`	`Admin@123`	`/admin/dashboard`
Approved Seller	`seller1@spmart.com`	`Seller@123`	`/seller/dashboard`
Customer (Buyer)	`buyer@spmart.com`	`Buyer@123`	`/account/profile`

10. Conclusion & Academic Evaluation

SP MART demonstrates the successful design, implementation, and deployment of an enterprise electronics e-commerce web application. By integrating Java 21 LTS and Spring Boot 3 with catalog-grounded AI recommendations and Progressive Web App mobility, the project solves real-world retail challenges, achieving exceptional response times, zero AI hallucinations, and comprehensive multi-role data governance.

_____ _____ _____

Candidate Signature **Internal Project Guide** **Head of Department (AI & DS)**